

206. 反转链表

反转链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy (75.75%)	3868	0	-	-	0

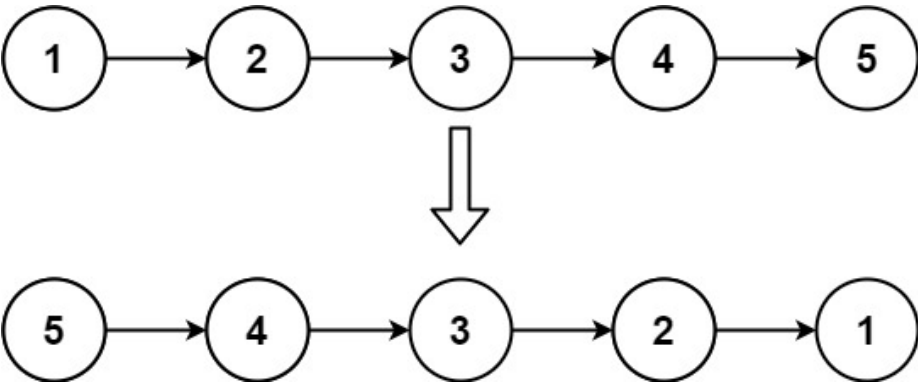
- Tags
- Companies

给你单链表的头节点

head

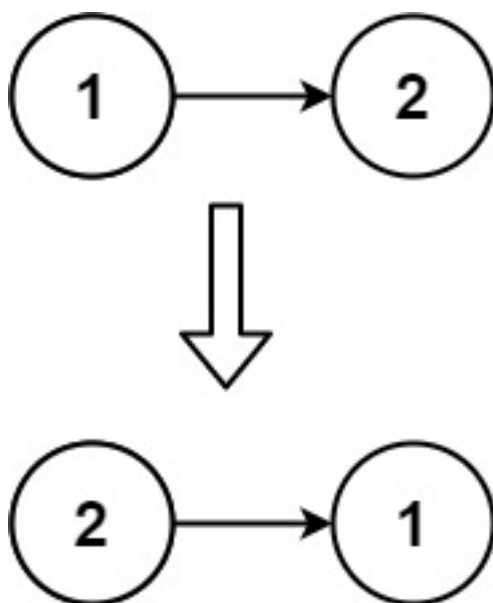
，请你反转链表，并返回反转后的链表。

示例 1:



输入: head = [1,2,3,4,5]
输出: [5,4,3,2,1]

示例 2:



输入: `head = [1,2]`

输出: `[2,1]`

示例 3:

输入: `head = []`

输出: `[]`

提示:

- 链表中节点的数目范围是 $[0, 5000]$
- $5000 \leq \text{Node.val} \leq 5000$

进阶: 链表可以选用迭代或递归方式完成反转。你能否用两种方法解决这道题?

Discussion | Solution

2. 方法

LeetCode 206: 反转链表 (详细解析)

1. 问题理解

目标: 反转一个单链表, 即将链表从头到尾的顺序颠倒过来。

输入: 单链表的头节点 `head`

输出: 反转后的链表的头节点

示例:

- 输入: [1,2,3,4,5]
- 输出: [5,4,3,2,1]

2. 基本思路

对于链表反转, 我们有两种主要方法:

1. 迭代法 - 使用循环逐个反转节点之间的连接
2. 递归法 - 利用递归的特性, 先反转后面的链表, 再处理当前节点

3. 解法一: 迭代法 (直观易懂)

思路

核心思想: 在遍历链表的过程中, 将当前节点的 **next** 指针改为指向前一个节点。由于节点没有引用其前一个节点, 所以需要事先存储其前一个节点。同时, 为了不丢失节点之间的连接, 还需要存储当前节点的下一个节点。

三个指针:

- **prev** - 指向前一个节点
- **curr** - 指向当前节点
- **next** - 临时变量, 存储当前节点的下一个节点

图解过程

假设原链表为 1->2->3->4->5:

1. 初始状态:

```
prev = null
curr = 1->2->3->4->5
```

2. 第一次迭代:

```
保存 next = 2->3->4->5
反转指针: 1->null
移动 prev: prev = 1->null
移动 curr: curr = 2->3->4->5
```

3. 第二次迭代:

```
保存 next = 3->4->5
反转指针: 2->1->null
移动 prev: prev = 2->1->null
移动 curr: curr = 3->4->5
```

4. 依此类推, 直到迭代完所有节点...

5. 最终状态:

```
prev = 5->4->3->2->1
curr = null
```

6. 返回 `prev` 作为新的头节点

代码实现

```
public ListNode reverseList(ListNode head) {
    // 定义前一个节点，初始为 null（因为第一个节点反转后，next 将指向 null）
    ListNode prev = null;
    // 当前节点，初始为头节点
    ListNode curr = head;

    // 遍历链表
    while (curr != null) {
        // 临时保存当前节点的下一个节点
        ListNode next = curr.next;

        // 反转当前节点的指针，使其指向前一个节点
        curr.next = prev;

        // 更新 prev 和 curr 指针，为下一次迭代做准备
        prev = curr;
        curr = next;
    }

    // prev 现在指向原链表的最后一个节点，即反转后的头节点
    return prev;
}
```

复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 是链表的长度，需要遍历整个链表一次
- 空间复杂度： $O(1)$ ，只使用了常数个额外变量，不随输入规模变化

4. 解法二：递归法（优雅但稍难理解）

思路

核心思想：假设链表的其余部分已经反转，现在我们应该如何反转它前面的部分？

假设链表为： $n1 \rightarrow n2 \rightarrow \dots \rightarrow nk-1 \rightarrow nk \rightarrow nk+1 \rightarrow \dots \rightarrow nm \rightarrow null$

若从节点 $nk+1$ 到 nm 已经反转，即变成 $\dots \leftarrow nk+1 \leftarrow \dots \leftarrow nm$

我们希望 $nk+1$ 的下一个节点指向 nk ，所以令 `nk.next.next = nk`

递归执行过程

以链表 1->2->3->4->5 为例:

1. 递归到最后节点 5:

head = 5, head.next = null, 返回 5 作为新头节点

2. 回溯到节点 4:

head = 4, newHead = 5

执行: 4.next.next = 4 // 即 5.next = 4

执行: 4.next = null // 断开原连接

返回: newHead(5)

链表变为: 5->4->null

3. 回溯到节点 3:

head = 3, newHead = 5

执行: 3.next.next = 3 // 即 4.next = 3

执行: 3.next = null // 断开原连接

返回: newHead(5)

链表变为: 5->4->3->null

4. 依此类推, 直到回溯完所有节点...

5. 最终返回 newHead(5) 作为新的头节点

代码实现

```
public ListNode reverseList(ListNode head) {  
    // 基本情况: 空链表或只有一个节点的链表, 直接返回  
    if (head == null || head.next == null) {  
        return head;  
    }  
  
    // 递归反转除了头节点之外的其余部分  
    ListNode newHead = reverseList(head.next);  
  
    // 将当前节点的下一个节点指向当前节点  
    // 例如: 对于 1->2->..., 执行后变成 2->1  
    head.next.next = head;  
  
    // 断开当前节点的原始 next 指针  
    head.next = null;  
  
    // 返回新的头节点 (原链表的最后一个节点)  
    return newHead;  
}
```

复杂度分析

- 时间复杂度: $O(n)$, 其中 n 是链表的长度, 每个节点只会被访问一次
- 空间复杂度: $O(n)$, 由于递归调用使用了调用栈, 最大调用深度为 n

5. 完整代码 (两种解法)

```
.java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    // 方法一: 迭代法
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            // 保存下一个节点, 防止链接断开后丢失
            ListNode next = curr.next;

            // 反转当前节点的指针
            curr.next = prev;

            // 移动指针, 为下一次迭代做准备
            prev = curr;
            curr = next;
        }

        // 返回新的头节点
        return prev;
    }

    // 方法二: 递归法
    /*
    public ListNode reverseList(ListNode head) {
        // 基本情况: 空链表或只有一个节点
        if (head == null || head.next == null) {
            return head;
        }
    }
    */
}
```

```

    }

    // 递归反转剩余部分
    ListNode newHead = reverseList(head.next);

    // 反转当前节点与下一个节点的连接
    head.next.next = head;
    head.next = null;

    // 返回反转后的头节点（即原链表的最后一个节点）
    return newHead;
}
*/
}

```

6. 如何在面试中讲解

1. 首先理解题目需求：
 - “我需要反转一个单链表，就是要把链表的指向完全颠倒过来。”
2. 提出两种方法：
 - “我可以用两种方法解决：迭代和递归。先讲迭代法，它更直观。”
3. 讲解迭代方法：
 - “迭代法的核心是使用三个指针：prev（前一个节点）、curr（当前节点）和 next（临时变量保存下一个节点）。”
 - “我们遍历链表，对每个节点执行：保存 next、反转指针、移动 prev 和 curr。”
 - “具体来说，对于每个节点，我们先存储它的下一个节点，然后将它的 next 指针指向前一个节点，然后移动 prev 和 curr 指针。”
4. 讲解递归方法：
 - “递归法则更加优雅但不太直观。假设后面的链表已经反转好了，我们只需要考虑如何反转当前节点。”
 - “对于节点 head，我们先递归反转 head.next 之后的所有节点，得到新的头节点 newHead。”
 - “然后将 head 的下一个节点的 next 指向 head 自己，即 head.next.next = head。”
 - “最后将 head 的 next 置为 null，断开原来的连接，并返回 newHead。”
5. 比较两种方法：
 - “迭代法空间复杂度更优，为 $O(1)$ ；递归法受递归栈的影响，空间复杂度为 $O(n)$ 。”
 - “两种方法的时间复杂度都是 $O(n)$ ，因为都需要遍历整个链表。”
6. 强调健壮性：
 - “两种实现都考虑了边界情况，如空链表或只有一个节点的链表。”

通过清晰解释这两种方法的思路和实现细节，可以展示对链表操作的深入理解。

找到具有 1 个许可证类型的类似代码